

Document number: P2004R0
Project: Programming Language C++
Audience: SG6, LEWGI

Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2019-12-17

Numbers and their Scopes

“A huge gap exists between what we know is possible with today's machines and what we have so far been able to finish.”

— Donald Knuth

I. Quick Introduction

This paper provides explanation on [P1889 "C++ Numerics Work In Progress"](#) and attempts to nail down scopes and priorities of P1889.

II. The main question

SG6 is doing a great job for many years. Right now SG6 has multiple papers with more than a 10 years history. SG6 planned to design and ship those papers as a whole (probably as a TS). [P0101 "An Outline of a C++ Numbers Technical Specification"](#) from year 2015 was attempting to put most of the papers under the single roof.

Last notes of [the P0101 discussion](#) are:

- the plan is to have a working draft at the end of the next meeting
- end of this week we should report a vision and outline

P1889 puts P0101 papers together. It is a starting point for the outcome of the P0101 discussion.

However, some concerns were expressed on the reflector. That leads us to the **main question**:

Are we willing to start the work on a united numbers paper P1889?

Some important points to consider

1. Many of the papers from P0101 are not maintained any more or do not evolve on their own.
2. People are not aware of the P0101 and keep designing, reviewing and evolving papers without a consistent view.
3. Many numeric papers are close to being ready to ship but can not progress because SG6 wish to ship them as a part of a bigger paper.
4. TSes are not very popular nowadays. Progressing individual papers works faster than TS. We need to came up with a **question** that TS must answer.

My personal opinion is following:

"WG21 can come up with new ideas and numeric papers every meeting. That's a good and never ending process. That process should not block existing/preapproved papers from shipping. Starting a united Numbers paper or abandoning the TS idea will help us to progress."

Tentative Polls:

1. Keep evolving papers independently forwarding ready papers to other subgroups, discarding the P0101 plans for a TS.
2. Approve the content of P1889R1 to be the "Numbers sketchbook" paper with a future plans to evolve it and merge into the C++ WD.

III. The Name and Motto

We need a short name and sentence that describes the P1889, so that any reader that is unaware of the P1889 could understand it's intent.

How about "Numbers" and "User friendly numeric types, aliases and low level building blocks to implement them"?

IV. The Scope

P0101 was providing the "Introduction on SG6" and "Design Principles" paragraphs. Those are a good starting points for understanding of the SG6 motivation for work that was done with the numeric papers.

For P1889 we need a clear understanding of scope and priorities. We need an understanding of "When to stop designing". Proposed scopes are:

1. Provide new numeric types that are widely useful and were already discussed
2. Provide basic building blocks for implementing new numeric types on top of build-in types
3. Provide minimal and consistent functionality to make the introduced types and functions usable for basic use-cases

Examples of fitting ideas:

1. Add unbounded integer type — it is a widely useful facility that is required for solving many mathematical tasks. It exists in almost any modern programming language. It's been discussed in SG6 and preapproved during the discussion of P0101
2. Add overflow detection and rounding facilities — those are important for implementing new numeric types. They've been discussed in SG6 and preapproved during the discussion of P0101
3. Add `to_chars/from_chars` overloads for all the numeric types — this is a minimal building block for `string<->number` conversions and without it users won't be able to output the number

Examples of non-fitting ideas:

1. Add binary-coded-decimal number type — that idea was not discussed in SG6 before and it is a very exotic number representation.
2. Add converters from binary-coded-decimal type to `int` — binary-coded-decimal type is not a built-in type.
3. Extend `std::locale` to localize new types — this is not a minimal required functionality. This could be added later.

V. Schedule

To progress we need a schedule. Or at least a deadline to stop adding new numeric types and concentrate on consistency and functionality.

VI. Acknowledgements

Many thanks to all the people who participate in the SG6.